Rent A Car

Курсов проект – Национална програма „Обучение за ИТ умения и кариера“

Уеб приложение за наемане на коли

Математическа гимназия „Академик Кирил Попов“
4001 Пловдив, ул. „Чемшир“ 11
тел.: + 359 32 643 157,
E – mail: https://omg-bg.com/

Ръководител: Росен Вълчев

Изготвили проекта:
Гергана Лозева
Деница Тренчева
Васил Пищялов

Съдържание

1. Цели на проекта

2. Разпределение на ролите

3. Ниво на сложност на проекта
   - 3.1. Многослойна архитектура
   - 3.2. Работа с база данни
   - 3.3. Уеб интерфейс и потребителско изживяване
   - 3.4. Работа с Git и GitHub
   - 3.5. Разширяемост на проекта

4. Инсталация и деинсталация

5. Основни етапи в реализирането на проекта

6. Реализация
   - 6.1. Архитектура
   - 6.2. База данни
   - 6.3. Описание на приложението
   - 6.4. Използвани технологии
   - 6.5. Използвани езици за програмиране

7. Система за удостоверяване и роли

8. Развитие и бъдещи подобрения

9. Използвана литература

1. Цели на проекта

Rent a Car е уеб базирана система, която служи за връзка между потребители и компания за отдаване на автомобили под наем. Системата е изградена на базата на ASP.NET Core 8 MVC архитектура с Microsoft SQL Server база от данни.

Основните цели на системата са:

- Лесно разглеждане на наличните автомобили
- Резервиране на автомобил за желан период
- Администриране на потребители, автомобили и резервации

2. Разпределение на ролите

Разработката на проекта е реализирана в екип, като задачите са разпределени между участниците – работа по backend логиката, базата данни, потребителския интерфейс и документацията.

3. Ниво на сложност на проекта

3.1. Многослойна архитектура

Проектът е реализиран по шаблона MVC (Model-View-Controller), който осигурява ясно разделение между бизнес логиката, данните и потребителския интерфейс. Контролерите обработват HTTP заявките, моделите описват структурата на данните, а изгледите (Razor .cshtml файлове) генерират HTML съдържанието.

3.2 Работа с база данни

Базата данни е реализирана с подхода Code First на Entity Framework Core. Това означава, че структурата на базата данни се дефинира чрез C# класове, а не ръчно написани SQL скриптове. Миграциите се прилагат автоматично при стартиране на приложението.

Реализирани са следните аспекти:

- Уникални индекси за полетата username, ЕГН и email
- Релации между таблиците с настроено поведение при изтриване
- Проверка за припокриване на резервационни периоди чрез LINQ заявка
- Защита от race condition при едновременно подаване на резервации

3.3. Уеб интерфейс и потребителско изживяване

Интерфейсът е изграден с Razor Views, HTML,CSS и Bootstrap.
Навигацията е интуитивна, като различните роли имат достъп до различни функционалности.

3.4. Работа с Git и GitHub

Проектът е разработен с използване на Git и GitHub, което позволява:

- контрол на версиите;
- екипна работа;
- проследяване на промените.

3.5 Разширяемост на проекта

Архитектурата на проекта позволява лесно добавяне на нови функционалности в бъдеще, като например качване на снимки за автомобилите, система за известия по имейл, филтриране на автомобили по марка и цена, и интеграция с платежна система.

4. Инсталация и деинсталация

4.1 Инсталация

1. Клониране на проекта от GitHub
2. Отваряне на **RentACar.sln**
3. Настройване на connection string в **appsettings.json**
4. Стартиране на миграциите
5. Стартиране на приложението

4.2 Деинсталация:

Изтриване на проекта и базата данни.

5. Основни етапи при реализирането на проекта

- Създаване на модели и контекст
- Имплементация на контролери
- Реализация на уеб интерфейс
- Добавяне на авторизация и роли

- Тестване и документация

6. Реализация

6.1. Архитектура

Проектът е ASP.NET Core MVC уеб приложение с ясно разделение на отговорностите между моделите, контролерите и изгледите.

Потокът на една HTTP заявка е следният: браузърът изпраща заявка → ASP.NET Core маршрутизира към съответния контролер → контролерът взема данни от базата чрез AppDbContext→ подава ги към View → View генерира HTML → отговорът се връща на браузъра.

6.2. База данни

Базата данни съдържа три таблици

- **Users**
- **Cars**
- **Reservations**

6.3. Описание на приложението

Вход в системата: Потребителите влизат чрез уникално потребителско име и парола.

Регистрация: Системата предлага бърза връзка към форма за регистрация за нови потребители.

Rent a Car

Наемете автомобил лесно и бързо.
Широк избор от автомобили на достъпни цени.

Регистрирай се | Вход

Вход в системата

Потребителско име

Въведете потребителско име

Парола

Въведете парола

Вход

Нямате акаунт? Регистрирайте се

Регистрация

Собствено име: Иван

Фамилно име: Иванов

Потребителско име: ivan123

Имейл: ivan@example.com

ЕГН: 0000000000

Телефон: +359888123456

Парола

Потвърди парола

Регистрирай се

Вече имате акаунт? Влезте тук

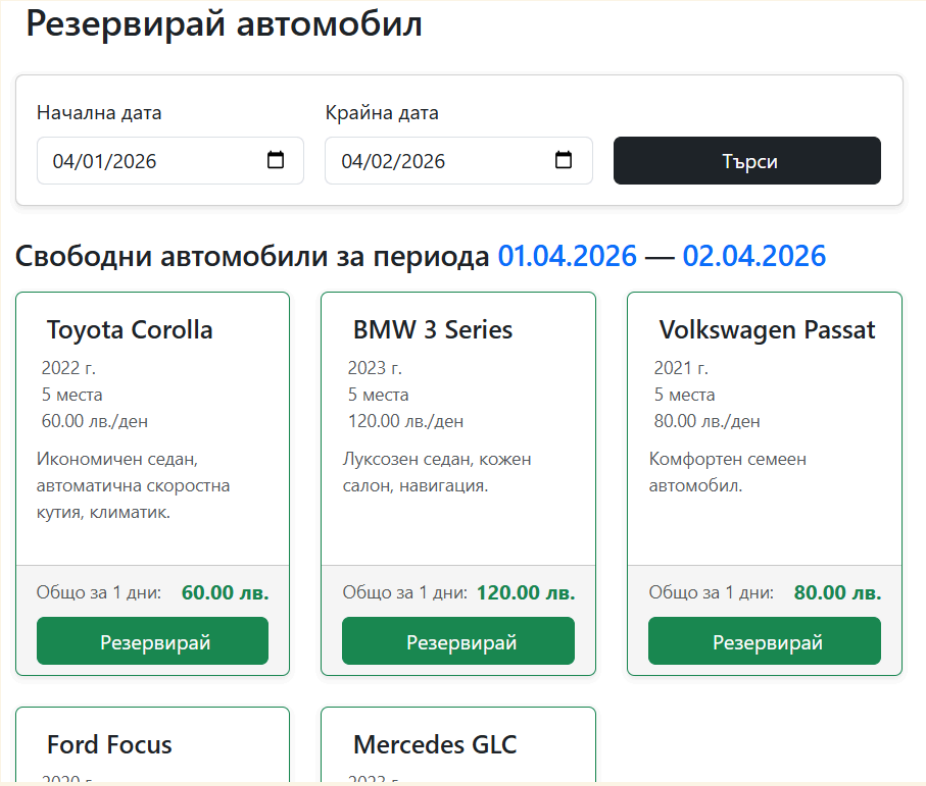
Контролери

AccountController — управлява автентикацията. При вход верифицира паролата с BCrypt и записва UserId, Username и Role в сесията. При регистрация проверява за уникалност на username, ЕГН и имейл преди да създаде акаунта.

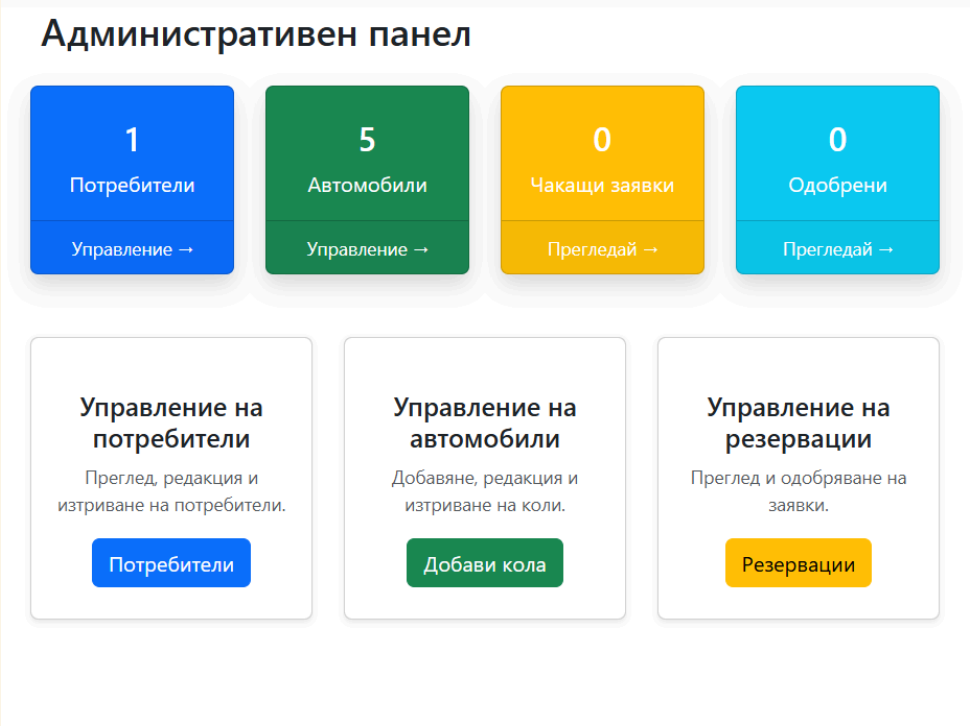
HomeController — зарежда началната страница с три произволни автомобила. Публично достъпен.

CarsController — CRUD операции за автомобили.Index и Details са публични, останалите действия (Create, Edit, Delete) изискват роля Admin.

ReservationsController — при търсене извършва LINQ заявка за автомобили без припокриващи се резервации. При подаване на резервация прави повторна проверка за наличност преди запис (защита от race condition). Изисква вход.



AdminController — пълно управление на потребители, автомобили и резервации. Зарежда статистика за таблото. Одобрява и отхвърля резервации. Изисква роля Admin.



Модели

User — представя потребител в системата. Има навигационно свойство към колекция от резервации.

Car — представя автомобил в автопарка. Има навигационно свойство към колекция от резервации.

Reservation — свързва потребител с автомобил за конкретен период. Има изчислено свойство,което не се съхранява в базата, а се изчислява като брой дни по цена на ден.

6.4. Използвани технологии

- ASP.NET Core MVC

  ASP.NET Core MVC е богата рамка за изграждане на уеб приложения и API, използвайки шаблона за проектиране Model-View-Controller.

Архитектурният модел Model-View-Controller (MVC) разделя приложението на три основни групи компоненти: Модели, Изгледи и Контролери. Този модел помага за постигане на разделяне на задачите . Използвайки този модел, потребителските заявки се насочват към Контролер, който е отговорен за работата с Модела за извършване на потребителски действия и/или извличане на резултати от заявки. Контролерът избира Изгледа, който да покаже на потребителя, и му предоставя всички данни за Модела, които са му необходими.

- Entity Framework Core

  Позволява на .NET разработчиците да работят с база данни, използвайки .NET обекти.Елиминира необходимостта от по-голямата част от кода за достъп до данни, който обикновено трябва да бъде написан.

  С EF Core достъпът до данни се осъществява с помощта на модел. Моделът е съставен от класове обекти и контекстен обект, който представлява сесия с базата данни. Контекстният обект позволява заявки и запазване на данни.

  EF поддържа следните подходи за разработване на модели:

  - ❖ Генериране на модел от съществуваща база данни.
  - ❖ Напишете ръчен код на модел, който да съответства на базата данни.
  - ❖ След като моделът е създаден, използвайте EF Migrations, за да създадете база данни от модела. Миграциите позволяват развитието на базата данни с промяната на модела.

- Microsoft SQL Server

  Microsoft SQL Server е собствена система за управление на релационни бази данни, разработена от Microsoft, използваща Structured Query Language (SQL). Като сървър за бази данни , той е софтуерен продукт с основната функция да съхранява и извлича

данни, изисквани от други софтуерни приложения , които могат да работят или на същия компютър, или на друг компютър в мрежа.

Представлява колекция от таблици с типизирани колони. Основният начин за извличане на данни от база данни на SQL Server е заявката за тях. Заявката декларативно указва какво трябва да се извлече. Тя се обработва от процесора за заявки, който определя последователността от стъпки, необходими за извличане на заявените данни.

- Bootstrap

  Библиотека за дизайн (CSS framework), която прави интерфейса на приложението модерен и отзивчив. Основната цел на добавянето ѝ към уеб проект е да се приложат изборите на Bootstrap за цвят, размер, шрифт и оформление към този проект.Благодарение на нея проектът изглежда добре както на компютър, така и на мобилни устройства.

- GitHub

  GitHub е собствена платформа за разработчици, която позволява на разработчиците да създават, съхраняват, управляват и споделят своя код. Тя използва Git, за да осигури разпределен контрол на версиите, а самият GitHub предоставя контрол на достъпа,проследяване на грешки,заявка за софтуерни функции управление на задачи,непрекъсната интеграция и уикита за всеки проект.

6.5. Използвани езици за програмиране

- **C#** – backend логика и модели

  Езикът C# е най-популярният език за платформата .NET , безплатна, междуплатформена среда за разработка с отворен код.C# е от семейството езици C. Синтаксисът на C# е познат, ако сте използвали C, C++, JavaScript, TypeScript или Java. Подобно на C и C++, точката и

запетаята (;) определят края на операторите. Идентификаторите на C# са чувствителни към главни и малки букви. C# използва по същия начин скоби, {както и }, управляващи оператори като if, else и switch, и циклични конструкции като for и while. C# също има foreach оператор за всеки тип колекция.

- **SQL** – чрез Entity Framework

- **HTML / CSS** – уеб интерфейс
  - ❖ HTML е стандартният език за маркиране на уеб страници. Уеб браузърите получават HTML документи от уеб сървър или от локално хранилище и ги превръщат в мултимедийни уеб страници. HTML описва семантично структурата на уеб страницата и първоначално включва сигнали за нейния външен вид.
  - ❖ CSS е основна технология на World Wide Web , наред с HTML и JavaScript. Проектиран е да позволи разделянето на съдържанието и представянето , включително оформление , цветове и шрифтове. CSS също така има правила за алтернативно форматиране, ако достъпът до съдържанието се осъществява на мобилно устройство .

- **JavaScript** – базова клиентска логика

  JavaScript е използван за поведение на уеб страниците от страна на клиента. Уеб браузърите имат специален JavaScript енджин , който изпълнява клиентския код. Тези енджин - и се използват и в някои сървъри и различни приложения . Най-популярната система за изпълнение за употреба извън браузър е Node.js. Той има динамично типизиране, обектно-базирана ориентация и първокласни функции. Той е мулти парадигмален, поддържайки стилове на програмиране, управлявани от събития, функционални и императивни. Разполага с интерфейси за приложно програмиране (API) за работа с текст, дати,

регулярни изрази, стандартни структури от данни и модела на обектите на документи (DOM).

7. Система за удостоверяване и роли

Системата за сигурност в приложението е реализирана чрез механизъм за удостоверяване чрез сесии (Session-based Authentication) и контрол на достъпа, базиран на роли (RBAC). Тя осигурява надеждна идентификация на потребителите и разграничаване на нивата на достъп до функционалностите на платформата.

1. Удостоверяване (Authentication)

Процесът по удостоверяване обхваща регистрацията и входа в системата:

Хеширане на пароли: За максимална сигурност паролите не се съхраняват в чист вид. Използва се алгоритъмът BCrypt, който генерира еднопосочен хеш при регистрация и извършва криптографска верификация при вход.

Управление на сесиите: След успешно удостоверяване, самоличността на потребителя се поддържа чрез HttpContext.Session. В сесията се съхраняват уникалният идентификатор (UserId), потребителското име и присвоената роля. Сесията е конфигурирана с време на живот от 60 минути и защита HttpOnly.

Валидация: Системата извършва задължителни проверки за уникалност на потребителско име, имейл и ЕГН, предотвратявайки дублиране на профили.

2. Роли и нива на достъп (Roles & Authorization)

Достъпът до ресурсите е дефиниран чрез две основни роли, записани в базата данни:

Потребител (User): Роля по подразбиране за всеки нов профил. Позволява достъп до стандартните клиентски функции (преглед на автомобили и резервации).

Администратор (Admin): Притежава разширени права за управление на системата. Само потребители с тази роля имат достъп до AdminController, където могат да:

Редактират и изтриват потребителски профили.

Променят статуса на резервации (Одобряване/Отхвърляне).

Сменят ролите на други потребители.

Контролът на достъпа се осъществява чрез логическа проверка на стойността Role в сесията на всяка заявка, изискваща административни привилегии.

8. Развитие и бъдещи подобрения

Функционални подобрения:

- Качване и показване на снимки за всеки автомобил
- Филтриране на автомобили по марка, цена и брой места
- Система за известия по имейл при одобрение/отхвърляне на резервация

9.Използвана литература

- https://docs.github.com/en
- https://learn.microsoft.com/en-us/docs/
- https://www.w3schools.com/

(последно посетени на 04.04.2026г.)